Guia tecnica del proyecto e indice del backend

Revision documental: 2026-10-29

Fuente editable: Markdown del repositorio. El indice contiene rutas, hashes, declaraciones y lineas, no cuerpos de fuentes ni credenciales. Este generador no ejecuta pruebas.

README.md

Sistema de administración de personas

Aplicación cliente-servidor para administrar titulares, direcciones múltiples, comunicaciones, contactos de emergencia compartidos y cuentas de acceso. Revisión documental: 29 de octubre de 2026. Esta documentación describe el código versionado; no certifica el estado de una base de datos ya instalada.

Documentación técnica

Documento	Contenido
Arquitectura (docs/ARQUITECTURA.md)	Capas, flujos, transacciones, autenticación y cachés
Contrato HTTP (docs/API.md)	Endpoints, DTOs, validaciones y respuestas
Persistencia (docs/BASE_DE_DATOS.md)	Tablas, restricciones, relaciones, índices y semilla
Operación y pruebas (docs/OPERACION_Y_PRUEBAS.md)	Instalación, configuración, ejecución, verificación y diagnóstico
Revisión técnica (docs/REVISION_TECNICA.md)	Inconsistencias, límites de la implementación y cobertura comprobada
Frontend (Frontend/README.md)	Pantallas, rutas, formularios, servicios y Angular Material
Guía de funcionamiento Angular (razi/Guia_Funcionamiento_Angular.docx)	Componentes, señales, RxJS, formularios y flujos
Detalle de migración Angular 22 (razi/Detalle_Migracion_Angular_22.docx)	Cambios por capa y componente; Angular 18 a 22, Vitest y animaciones
Backend (Backend/demo/README.md)	Paquetes Java, reglas de negocio y generación de documentación
Guía PDF del backend (docs/back/Guia_backend_archivos_y_lineas.pdf)	Documentación técnica exportada e índice de fuentes con líneas
Generador y manifiesto del PDF (docs/back/generar_guia_backend.py)	Generación reproducible e inventario verificable de fuentes
Especificación frontend (docs/front/FRONTEND_ESPECIFICACION_AGENTES.md) · backend (docs/back/BACKEND_ESPECIFICACION_AGENTES.md)	Invariantes y contexto técnico para cada parte

Organización del repositorio

```
  docs/                                Toda la documentación y artefactos documentales
    back/                              Documentación y referencias específicas del backend
    front/                             Documentación específica del frontend
razi/Guia_Funcionamiento_Angular.docx Guía explicada del frontend Angular
razi/Detalle_Migracion_Angular_22.docx Informe de migración Angular 22
Frontend/
  src/app/                           Aplicación Angular standalone
  src/styles.scss                    Entrada global de estilos
  src/material-typography.scss        Tema Material 3 y tipografía
  angular.json                       Build, servidor y pruebas
Backend/demo/
  src/main/java/com/example/demo/     API Spring y persistencia JPA
  src/main/resources/schema.sql      Esquema para una base nueva
  src/main/resources/data.sql        Semilla de desarrollo
  src/test/java/                     Pruebas unitarias e integración
  scripts/HashPassword.java          Utilidad interactiva BCrypt
```

Tecnologías comprobadas

Capa	Dependencias y configuración
Frontend	Angular/Material/CDK 22.x, TypeScript 6.x, RxJS 7.8.x y Zone.js 0.16.x; versiones exactas en package-lock.json
Compilación web	Angular CLI con @angular/build, Sass y Vitest/jsdom para pruebas
Backend	Java 21, Spring Boot 4.1.1, Spring MVC, Security, Data JPA y JDBC
Autenticación	JJWT 0.11.5, HS256; contraseñas BCrypt
Persistencia	PostgreSQL, SQL nativo con funciones específicas de PostgreSQL
Herramientas	Maven Wrapper 3.9.16; generador Python con ReportLab y pypdf

El proyecto no declara una versión concreta de servidor PostgreSQL, contenedores, pipeline CI ni pruebas E2E.

Inicio local

1. Crear una base vacía y ejecutar schema.sql seguido de data.sql, una sola vez. Ver procedimiento y límites (docs/OPERACION_Y_PRUEBAS.md).

2. Configurar la conexión y la clave de firma del backend sin copiar sus valores a la documentación.

3. Desde Backend/demo, ejecutar mvnw.cmd spring-boot:run.

4. Desde Frontend, ejecutar npm.cmd ci y npm.cmd start.

El frontend espera http://localhost:8080/api/ y se sirve normalmente en http://localhost:4200. Las URLs están codificadas en los servicios y en el interceptor, no centralizadas en un archivo de entorno.

Modelo funcional vigente

- Una persona es titular cuando tiene una fila en perfil_titular; no existe persona.es_titular.

- Un titular activo tiene perfil_titular.fecha_baja IS NULL; el límite de servicio es 20 titulares activos.

- Las direcciones se almacenan exclusivamente en direccion_titular; no existen columnas direccion ni ciudad en perfil_titular.

- Cada alta necesita al menos una dirección, un correo, un teléfono y un contacto de emergencia. Las direcciones tienen un máximo de 20; los contactos no tienen máximo explícito.

- El parentesco pertenece a la relación titular/contacto. Editar los datos personales del contacto modifica la persona compartida.

- El listado transporta un resumen; los detalles y el formulario completo tienen contratos independientes.

- La sesión se limpia ante 401, fallo de conexión, respuesta 5xx o timeout de la API local. La detección ocurre al realizar una petición; no hay monitor periódico del backend.

Las restricciones del formulario, del servicio Java y del SQL no son idénticas. Las diferencias están registradas en la revisión técnica (docs/REVISION_TECNICA.md), sin modificar reglas de negocio durante esta actualización documental.

docs/API.md

Contrato HTTP vigente

Convenciones

Base local: http://localhost:8080. Los endpoints de negocio requieren Authorization: Bearer <JWT>; el único endpoint de aplicación permitido sin autenticación es /api/auth/login. OPTIONS está permitido globalmente. CORS declara http://localhost:4200 en los controllers. No hay versionado /v1, paginación ni documentación OpenAPI generada.

Los IDs JSON corresponden a Long, las fechas a LocalDate (YYYY-MM-DD) y las comunicaciones a arreglos de cadenas. Este documento usa nombres y tipos; no contiene credenciales ni registros personales de la semilla.

Endpoints

Método	Ruta	Entrada	Respuesta exitosa
POST	/api/auth/login	username, password	200: message, username, token, usuarioActual
GET	/api/auth/me	Identidad del JWT	200: UsuarioActualDTO
GET	/api/auth/me/login	Identidad del JWT	200: {username:string}
PUT	/api/auth/me/login	EditarCuentaDTO	200: {ok:true}
GET	/api/personas	—	200: PersonaResumenDTO[] de activos
GET	/api/personas/inactivos	—	200: PersonaResumenDTO[] de dados de baja
GET	/api/personas/resumen	—	200: ResumenDTO
GET	/api/personas/{id}	ID titular activo	200: PersonaDTO completo para editar
GET	/api/personas/{id}/detalles	ID titular activo	200: PersonaDetalleDTO
POST	/api/personas	PersonaDTO con contactosEmergencia	201: {ok:true}
PATCH	/api/personas/{id}	Objeto parcial con campos modificados	200: PersonaActualizadaDTO (ok y objeto parcial persona)
PUT	/api/personas/{id}	PersonaDTO completo, sin sustituir contactos	200: PersonaActualizadaDTO (ok y fila persona); compatibilidad
DELETE	/api/personas/{id}	ID titular activo	204, sin cuerpo; baja lógica
PUT	/api/personas/{id}/reactivar	Sin cuerpo útil	200: {ok:true}
GET	/api/personas/{id}/contactos	ID titular activo	200: ContactoDTO[]; header Server-Timing
PUT	/api/personas/{id}/contactos	ContactoDTO[] completo	200: {ok:true}
POST	/api/personas/contactos/coincidencias	BusquedaContactoDTO	200: ContactoDTO[] de candidatos
GET	/api/ocupaciones	—	200: objetos {id,nombre}
GET	/api/parentescos	—	200: objetos {id,nombre}, ordenados por ID

PUT /api/personas/{id} devuelve {ok:true,persona:PersonaResumenDTO} construido dentro de la misma transacción. El frontend reemplaza esa fila por ID y no hace otro GET del listado ni del formulario para reflejar el guardado. Los detalles almacenados de esa persona se invalidan; los de los demás permanecen. usuarioActual.idPersona, tanto en login como en /auth/me, identifica de forma inequívoca la cuenta del encabezado.

POST /api/personas crea una unidad transaccional y no devuelve su ID. La UI vuelve al listado después de guardar. No hay endpoint de alta de usuario, refresco/revocación JWT, salud del backend, borrado físico de titular.

DTOs de lectura

```
type Direccion = {
  pais: string; estado: string; municipio: string; colonia: string;
  codigoPostal: string; calle: string; numero: string;
};
type PersonaResumenDTO = {
  id: number; nombre: string; apellido: string; ocupacion: string;
```

```
  correos: string[]; telefonos: string[]; fechaBaja: string | null;
  direcciones: { municipio: string }[]; puedeEliminar: boolean;
};
type PersonaDetalleDTO = {
  genero: string; fechaNacimiento: string; direcciones: Direccion[];
};
type PersonaDTO = {
  id: number | null; nombre: string; apellido: string;
  fechaNacimiento: string; genero: string; ocupacion: string;
  fechaBaja: string | null; correos: string[]; telefonos: string[];
  direcciones: Direccion[]; contactosEmergencia?: ContactoDTO[];
};
type ContactoDTO = {
  idContacto: number | null; nombre: string | null; apellido: string | null;
  fechaNacimiento: string | null; genero: string | null;
  correos: string[] | null; telefonos: string[] | null;
  idParentesco: number | null; parentesco: string | null;
};
type PersonaActualizadaDTO = { ok: boolean; persona: PersonaResumenDTO };
type UsuarioActualDTO = { idPersona: number; nombre: string; correo: string | null };
type ResumenDTO = {
  total: number; conCorreo: number; conTelefono: number;
  ocupaciones: { nombre: string; cantidad: number }[];
};
```

Los tipos anteriores describen respuestas y nulabilidad del código; no son un esquema de validación automático. PersonaDTO.contactosEmergencia se omite al ser null y no se rellena en obtenerPorId. Los contactos se leen por su ruta independiente. No existen campos JSON direccion, ciudad, correo singular ni telefono singular para titulares.

El listado devuelve solo la primera dirección, reducida a municipio; el arreglo puede quedar vacío. Los detalles incluyen todas las direcciones ordenadas por orden ASC, id ASC y no incluyen nombre, apellido, ocupación ni comunicaciones. puedeEliminar es false cuando existe un usuario vinculado con rol=ROLE_ADMIN; es una ayuda a la UI y la baja vuelve a comprobarlo en el servicio.

El resumen agrega titulares activos. conCorreo/conTelefono cuentan existencia de al menos una comunicación. ocupaciones es top 5, orden por cantidad descendente y nombre como desempate. En contactos, Server-Timing tiene formato contactos;dur=<milisegundos> y mide la llamada de servicio, no el tiempo completo de red. CORS no declara ese header como exposed header; verlo en DevTools no implica que HttpClient pueda leerlo desde otro origen.

Alta y actualización de titular

Campos efectivos de escritura: nombre, apellido, fechaNacimiento, genero, ocupacion, correos, telefonos, direcciones; contactosEmergencia solo interviene en alta. ID y baja no se toman del body para estas operaciones. aceptaTerminos, controles de nombres separados y cp no pertenecen al DTO.

```
| Campo/regla | Validación explícita del servicio |
| --- | --- |
| nombre, apellido | No nulos/blank, hasta 100 caracteres; no se recortan al asignar titular |
| fechaNacimiento | No nula; no hay regla de edad ni prohibición de fechas futuras |
| genero titular | Se asigna, sin validación explícita; SQL requiere no null y máximo 50 |
| ocupacion alta | Obligatoria y no blank; se busca por nombre y se crea si no existe |
| ocupacion edición | Si es no nula/no blank se busca o crea; si se omite se conserva |
| correos | Al menos uno; no null; hasta 150; regex ^[^\s@]+@[^\s@]+\.[^\s@]+$; unicidad de la lista después
de trim/lowercase |
| telefonos | Al menos uno; cadenas de exactamente diez dígitos; sin repetidos |
```

| direcciones | Entre 1 y 20 objetos no nulos; escritura reemplaza la colección completa |

| calle | No null/blank, hasta 255; almacenamiento con trim |

| municipio | No null/blank, hasta 100; almacenamiento con trim |

| pais, estado, numero | Hasta 100; null/blank se normaliza (pais a México, demás a vacío) |

| colonia | Hasta 150; null/blank a vacío |

| codigoPostal | Vacío/null o exactamente cinco dígitos; hasta 5 |

Los correos se almacenan con trim y minúsculas; teléfonos con trim. La regex de validación del titular se aplica antes de normalizar, por lo que los espacios alrededor de un correo pueden rechazarse. La unicidad SQL es por persona, no global entre todas las personas. El frontend exige todos los campos de dirección, incluido CP y número; el backend admite algunos vacíos.

Contactos compartidos y sustitución de relaciones

PUT /api/personas/{id}/contactos recibe el conjunto completo deseado. IDs omitidos se desvinculan; los presentes se conservan; nuevos se agregan. El conjunto no puede estar vacío y no tiene máximo explícito.

| Modo | Campos de entrada | Efecto |

| --- | --- | --- |

| Nuevo | Sin idContacto; personales y comunicaciones completos; parentesco | Crea Persona sin perfil titular y vincula |

| Reutilizado | idContacto positivo y idParentesco o nombre de parentesco | Cambia/crea solo la relación; preserva datos personales |

| Actualización compartida | ID y todos los campos personales/comunicaciones | Modifica la misma persona; afecta sus otros titulares |

Enviar algún campo personal con un ID hace obligatorios todos los datos personales. Nombre y apellido: no blank, máximo 100; fecha: obligatoria; género: no blank. Correos y teléfonos cumplen las mismas reglas anteriores. No hay validación explícita de longitud 50 del género del contacto, pero SQL sí la impone.

IDs positivos y únicos; no autorreferencia. El contacto existente debe no tener perfil o tener perfil activo. Parentesco: ID positivo existente o nombre exacto no blank de hasta 50; no se crea catálogo. Correos/teléfonos enviados en distintos contactos de la solicitud no pueden repetirse. La búsqueda de coincidencias excluye el ID que se está actualizando y rechaza coincidencias sin confirmación/reutilización con 409.

Al quitar la última referencia se elimina físicamente la persona de contacto solo si no tiene perfil titular, cuenta ni vínculos entrantes/salientes. El DELETE SQL conserva personas compartidas o protegidas. Sus correos/teléfonos se eliminan por ON DELETE CASCADE.

Coincidencias

Entrada: {correos?:string[], telefonos?:string[], excluirId?:number}. Requiere al menos un valor; valida formato correo y teléfono. Busca por OR entre comunicaciones, normalizando correos. excluirId excluye la propia persona. Candidatos: personas referenciadas como contacto, o personas sin perfil ni cuenta. No es un buscador universal de cualquier titular/administrador. El DTO de candidato lleva personales y comunicaciones, con parentesco/ID de parentesco null.

Cuenta y login

Login no verifica baja ni rol. Devuelve 404 si no encuentra username y 401 si BCrypt no coincide. El body es Map, sin validación Bean Validation explícita; entradas incompletas pueden terminar en errores distintos a esos dos casos.

EditarCuentaDTO: username, passwordActual, passwordNueva. Se exige username de 1–255 tras trim, sin espacios; contraseña actual válida; nueva opcional, mínimo 8 caracteres y máximo 72 bytes UTF-8. Username ocupado: 409. El SQL limita username a 50; esta discrepancia está registrada. Guardado exitoso no devuelve un token nuevo; el frontend cierra sesión y pide iniciar de nuevo.

Errores y límites HTTP

| Camino | Tratamiento implementado |

| --- | --- |

| Login | 404/401 con objeto {message} |

| Validaciones de servicio | ResponseStatusException (normalmente 400); conflictos 409; titular no encontrado 404 |

| POST personas | Conserva código de ResponseStatusException; IllegalArgumentException/RuntimeException se traducen a 400 con texto |

| PUT/DELETE persona | Conserva ResponseStatusException; IllegalArgumentException a 400; otras RuntimeException a 404 con texto |

| GET listado activo | Conserva ResponseStatusException; IllegalArgumentException 400; RuntimeException 500 con texto |

| Reactivar | 404 si no está dado de baja, 409 si hay 20 activos; otros fallos por manejo de Spring |

| Baja administrador | 403, sin cambiar su fecha de baja |

| GET por ID/detalles/contactos, PUT contactos, cuenta | Excepciones se propagan al manejo de Spring; cuerpo final depende de esa configuración |

No se garantiza un JSON uniforme de errores ni un código uniforme para fallos de persistencia. Un 404 en PUT/DELETE puede ocultar un fallo interno traducido por el controller. El interceptor frontend cierra sesión ante 401, 0, timeout y 5xx de la API, pero no ante 403 ni errores de validación. Las pruebas unitarias no sustituyen una prueba HTTP real de filtros, serialización y base.

Edicion parcial del titular

PATCH /api/personas/{id} requiere JWT. El frontend compara contra el formulario original en memoria y envia exclusivamente campos modificados: nombre, apellido, genero, fechaNacimiento, ocupacion, correos, telefonos, direcciones. Los campos omitidos se conservan; null explicito se rechaza. Las colecciones se sustituyen completas solamente cuando cambian.

Ejemplo de peticion: { "nombre": "Nuevo nombre" }. Respuesta: { "ok": true, "persona": { "nombre": "Nuevo nombre" } }. persona es un objeto parcial con los valores persistidos de los campos enviados; no incluye el resto del formulario ni repite la fila completa. PUT completo sigue disponible por compatibilidad.

La respuesta se fusiona por ID con la lista y los detalles en memoria. No hay GET posterior al guardar. Una edicion sin cambios no genera peticion. El encabezado se actualiza localmente solo cuando cambia nombre completo o correo principal de la cuenta activa.

docs/ARQUITECTURA.md

Arquitectura y flujos internos

1. Topología

```
flowchart LR
  UI[Angular standalone] --> HTTP[HttpClient / jwtInterceptor]
  HTTP --> SEC[Spring Security / JwtFilter]
  SEC --> CTRL[Controllers REST]
  CTRL --> SVC[PersonaService / CuentaService]
  SVC --> REPO[Spring Data JPA / SQL nativo]
  REPO --> DB[(PostgreSQL)]
  UI --> CP[Servicio externo de códigos postales]
```

El frontend es una SPA con estado en memoria y almacenamiento del navegador. No hay SSR, backend proxy para códigos postales, WebSocket ni bus de eventos entre clientes. Los servicios raíz Angular permanecen vivos durante la navegación; una recarga destruye sus cachés.

2. Frontend

app.config.ts registra Router, detección de cambios por Zone y HttpClient con transporte XHR e interceptor funcional JWT. Las animaciones legacy se retiraron; Inicio y Dashboard usan animate.enter y CSS. Los componentes son standalone y declaran sus módulos Material. Los formularios de registro/contactos/cuenta son reactivos; el login usa ngModel. Las señales controlan carga, errores, selección y datos visibles; RxJS gestiona HTTP, deduplicación y comprobación de coincidencias.

AppComponent contiene RouterOutlet y el feedback global. Escucha AuthService.tokenChanges$ y redirige a /inicio al perder el token fuera del login. AdminLayoutComponent proporciona sidenav, toolbar y datos de la cuenta. El menú es superpuesto hasta 760 px y persistente en escritorio. Sus subapartados se calculan desde la URL de edición de personas y contactos.

Los componentes convierten controles de formulario a DTOs; no deben enviarse directamente FormGroup.value ni propiedades visuales. Las fechas viajan como cadenas YYYY-MM-DD; los IDs son number en TypeScript y Long en Java.

3. Backend

```
| Paquete | Responsabilidad |
| --- | --- |
| controller | Mapping HTTP, cuerpos de respuesta, algunos mapeos manuales de excepciones y CORS |
| service | Validación explícita, coordinación de entidades, transacciones y construcción de DTOs |
| repository | CRUD JPA, búsqueda por comunicaciones, agregados, bloqueo y limpieza SQL |
| model | Entidades y relaciones persistentes |
| dto | Contratos de lectura/escritura; records excepto PersonaDTO |
| security | Firma/lectura JWT y población de SecurityContextHolder |
| config | Cadena de filtros, CORS, BCrypt y sesión stateless |
```

No existe @ControllerAdvice propio ni un formato único de error. Las entidades de catálogo se exponen directamente; las personas se representan mediante DTOs. No hay paginación en listados. El límite funcional de titulares reduce el volumen, pero no equivale a una garantía SQL.

4. Persistencia y transacciones

Persona contiene los datos comunes, correos, teléfonos y relaciones salientes. PerfilTitular usa PK compartida mediante @MapsId; almacena ocupación, baja y colección de direcciones. El contacto es otra Persona, no un objeto embebido con datos duplicados.

Las colecciones de personas usan cascada y orphanRemoval; en relaciones de emergencia, borrar el vínculo no aplica cascada REMOVE a la persona referenciada. La limpieza explícita ejecuta eliminarContactoSinReferencias después de saveAndFlush, dentro de la transacción.

Operaciones de escritura de PersonaService y CuentaService usan @Transactional; las lecturas principales usan readOnly=true. Ante una excepción no controlada dentro de una transacción se revierte la unidad de trabajo. Las escrituras confirman el resultado; PUT del titular también devuelve su PersonaResumenDTO para actualizar una fila sin consultar el listado completo.

guardar y guardarContactos adquieren pg_advisory_xact_lock(74192301): un bloqueo global transaccional para serializar esos caminos de alta/sustitución/limpieza. No está presente en reactivar ni actualizar. Por tanto, no se debe describir el límite de 20 como protegido en todos los caminos concurrentes.

Las lecturas de listado usan consultas JPA de entidades y después DTOs; el resumen de red es reducido, pero no es una proyección SQL exclusiva de esas columnas. Correos y teléfonos usan @BatchSize(32). La consulta de contactos evita ensamblar el grafo mediante SQL nativo y string_agg(..., chr(31)), con orden por IDs. Los agregados del dashboard usan FILTER, EXISTS y top 5 por ocupación.

5. Alta y edición

```
sequenceDiagram
  participant F as RegistroComponent
  participant C as ContactosComponent
  participant A as API
  participant D as PostgreSQL
  F->>F: Validar titular, direcciones y comunicaciones
  F->>F: Guardar borrador en memoria/sessionStorage
  F->>C: Navegar a /contactos
  C->>A: Consultar parentescos y coincidencias
  C->>A: POST /api/personas (titular + contactos)
  A->>D: Transacción: validar, crear/reutilizar, vincular
  D-->>A: Commit
  A-->>C: 201 {ok:true}
  C->>C: Limpiar borrador y volver a Personas
```

El primer paso del alta no crea una persona. El borrador usa la clave registro_borrador; JSON inválido se descarta. Logout elimina el borrador.

En edición, /registro/:id consulta GET /api/personas/{id} y bloquea guardado hasta cargar correctamente. Los campos nombres, apellidoPaterno y apellidoMaterno se normalizan a nombre y apellido; la división de apellidos usa espacios y no modela apellidos compuestos como entidades separadas. cp se transforma en codigoPostal.

La edición de un contacto conserva la lista original de vínculos y cambia el seleccionado. Los demás se envían como {idContacto,idParentesco}. Agregar concatena los nuevos a la lista original. Quitar envía la lista completa restante; no hay DELETE individual de contacto.

Las búsquedas de coincidencias usan debounceTime(350) y switchMap, y controlan revisiones para descartar respuestas/confirmaciones obsoletas. La reutilización confirmada bloquea los datos personales y envía solo ID/parentesco. Editar una persona compartida requiere confirmación y datos completos. El parentesco de los demás titulares no cambia.

6. Autenticación y resiliencia

El login busca Usuario por username, compara BCrypt y genera JWT HS256 con sub=username, iat y exp a una hora. JwtFilter verifica firma/vigencia al analizar el token y establece autenticación con autoridades vacías. No recarga la cuenta ni evalúa su rol o baja en cada petición.

AuthService almacena el JWT en localStorage, expone un BehaviorSubject y mantiene el perfil del header en una señal. La comprobación local decodifica exp; no verifica criptográficamente la firma. Los guards dependen de esa comprobación; la autoridad final es el backend.

Hay tres mecanismos de invalidación local: temporizador de vencimiento, evento storage y revisión del token en localStorage cada 250 ms. Este intervalo no realiza HTTP. Cambiar/cerrar sesión limpia perfil, consultas de cuenta, borrador y cachés de PersonasService.

El perfil se usa desde el login; al recargar, /api/auth/me lo recupera. Los cambios emiten datosPersonalesActualizados$ con el ID afectado. AuthService compara ese ID con usuarioActual.idPersona y comprueba nombre completo/correo principal. Para un titular combina los campos persistidos del PATCH y aplica el perfil localmente, sin GET /auth/me; para un contacto compartido solo consulta el perfil si el cambio personal corresponde a la cuenta activa. Cambios de otra persona, dirección, teléfono, ocupación o correo secundario no refrescan el encabezado. revisionCuenta y comparación de token evitan que una respuesta anterior sobrescriba el perfil vigente.

jwtInterceptor afecta únicamente a URLs cuyo prefijo es http://localhost:8080/api/. Añade Bearer cuando existe token, aplica timeout({each:15000}) y convierte TimeoutError a HttpErrorResponse con estado 0. Ante 401, 0 o 500–599 cierra sesión si el token actual coincide con el capturado al iniciar la solicitud. Propaga el error al consumidor. 400/403/404/409/422 no cierran sesión. Una caída no se detecta mientras no haya peticiones, y una respuesta obtenida de caché no prueba disponibilidad del backend.

7. Política de cachés

Consulta	Persistencia	Deduplicación e invalidación
Listado activo/inactivo y /{id}/detalles	Map en memoria, sin TTL	shareReplay(1), refCount:false; edición de titular mantiene listados con una capa de filas actualizadas por ID y combina los campos modificados de sus detalles; alta, baja, reactivación, contactos y sesión limpian la caché completa; errores eliminan la entrada
/{id} para editar	Solo mientras la petición está pendiente	Map solicitudes, finalize elimina URL, refCount:true; no conserva la respuesta para futuras consultas
/resumen y /ocupaciones	Solo solicitudes pendientes	Mismo mecanismo anterior
/{id}/contactos	TTL de 30 s desde creación de la entrada	shareReplay, refCount:false; se limpia al actualizar personas/contactos o sesión; error elimina entrada
/parentescos	Memoria por sesión	shareReplay, sin TTL; se limpia al cambiar sesión o fallar
/auth/me	Señal del perfil y observable compartido	Revisión de cuenta/token al actualizar; limpia al cambiar sesión

La capa filasActualizadas mantiene solo la versión más reciente por ID y se aplica al leer listados; evita cadenas de observables acumuladas y también cubre peticiones iniciales que terminen después del guardado. Las respuestas de escrituras iniciadas en otra sesión se descartan mediante revisionSesion. La señal del perfil compara ID/nombre/correo y no notifica por objetos equivalentes.

No hay IndexedDB, Service Worker, sincronización en tiempo real ni invalidación entre navegadores distintos. La invalidación elimina referencias del Map; no cancela automáticamente observables HTTP anteriores. Ver los límites de concurrencia y caché en la revisión técnica.

8. Diseño y accesibilidad

styles.scss importa el tema con @use './material-typography'. El tema Material 3 mantiene Azure/Blue, agrega Sora para títulos y Plus Jakarta Sans para texto, e incluye solo los módulos usados. Las variables --app-font-* unifican estilos locales; IBM Plex Mono se reserva para estadísticas y campos numéricos. index.html carga Google Fonts con display=swap, preconnect y familias de respaldo.

El feedback usa señales, avisos temporizados de 5 s y <dialog> nativo con showModal(), no MatDialog. El login y dashboard usan animate.enter con CSS y respetan prefers-reduced-motion. Formularios incluyen etiquetas, errores, estados de carga y atributos ARIA. Hay vistas móviles para Personas, Desactivadas y Contactos; la revisión de código no equivale a una auditoría de accesibilidad completa.

La edición del titular usa PATCH /api/personas/{id}. PersonasService conserva una copia del formulario cargado, compara los campos efectivos, omite la solicitud cuando no hay diferencias y envía las colecciones completas cuando cambian. La respuesta incluye los valores persistidos de los campos enviados y el servicio aplica deltas por ID al listado y detalles en memoria sin un GET adicional. El endpoint PUT completo sigue disponible para compatibilidad.

docs/BASE_DE_DATOS.md

Modelo PostgreSQL y persistencia JPA

Fuente de verdad

La estructura documentada procede de Backend/demo/src/main/resources/schema.sql, contrastada con las entidades. El script usa CREATE TABLE IF NOT EXISTS: define tablas nuevas y no convierte una base de una versión anterior. No existe un sistema activo de migraciones Flyway/Liquibase ni una carpeta de migraciones SQL vigente.

Las direcciones se almacenan solamente en direccion_titular. perfil_titular no tiene direccion ni ciudad. La existencia del perfil distingue titular/contacto; Persona.isTitular() es un método derivado, no una columna.

Relaciones

```
erDiagram
  PERSONA ||--o| PERFIL_TITULAR : tiene
  CATALOGO_OCUPACION ||--o{ PERFIL_TITULAR : clasifica
  PERFIL_TITULAR ||--o{ DIRECCION_TITULAR : direcciones
  PERSONA ||--o{ PERSONA_CORREO : correos
  PERSONA ||--o{ PERSONA_TELEFONO : telefonos
  PERSONA ||--o| USUARIO : cuenta
  PERSONA ||--o{ PERSONA_CONTACTO_EMERGENCIA : titular
  PERSONA ||--o{ PERSONA_CONTACTO_EMERGENCIA : contacto
  CATALOGO_PARENTESCO ||--o{ PERSONA_CONTACTO_EMERGENCIA : parentesco
```

El SQL permite cero direcciones y cero contactos por perfil. Los mínimos de negocio se imponen en el servicio, no por esas cardinalidades SQL. Los vínculos son dirigidos; compartir persona no implica compartir parentesco.

Diccionario de tablas

persona

Columna	Tipo	Restricción
id	BIGSERIAL	PK
nombre, apellido	VARCHAR(100)	NOT NULL
fecha_nacimiento	DATE	NOT NULL
genero	VARCHAR(50)	NOT NULL

Contiene datos comunes a titulares, contactos y administradores. No incluye comunicaciones, ocupación, baja ni indicadores de titular.

perfil_titular

Columna	Tipo	Restricción
id_persona	BIGINT	PK y FK a persona(id), ON DELETE CASCADE
id_ocupacion	BIGINT	NOT NULL; FK a catalogo_ocupacion(id)
fecha_baja	DATE	Nullable; NULL significa activo

PerfilTitular usa @MapsId; la identidad coincide con Persona.id. Una baja no elimina las comunicaciones, direcciones ni relaciones.

direccion_titular

Columna	Tipo	Restricción/default
id	BIGSERIAL	PK

| id_persona | BIGINT | NOT NULL; FK a perfil, ON DELETE CASCADE |

| orden | INTEGER | NOT NULL |

| pais | VARCHAR(100) | NOT NULL; default México |

| estado | VARCHAR(100) | NOT NULL; default vacío |

| municipio | VARCHAR(100) | NOT NULL |

| colonia | VARCHAR(150) | NOT NULL; default vacío |

| codigo_postal | VARCHAR(5) | NOT NULL; default vacío; vacío o cinco dígitos |

| calle | VARCHAR(255) | NOT NULL |

| numero | VARCHAR(100) | NOT NULL; default vacío |

ck_direccion_codigo_postal usa regex PostgreSQL ^[0-9]{5}$. Orden cero identifica la dirección principal. JPA ordena por orden ASC, id ASC. El servicio asigna orden desde la posición en el arreglo y reemplaza la colección; no expone un ID de dirección en el DTO. No hay UNIQUE (id_persona,orden), CHECK de orden no negativo, máximo de 20 ni mínimo de una dirección en SQL.

Comunicaciones

| Tabla | Columnas | Restricciones |

| --- | --- | --- |

| persona_correo | id BIGSERIAL, id_persona BIGINT, correo VARCHAR(150) | PK; FK CASCADE; correo/ID persona

NOT NULL; UNIQUE (id_persona,correo) |

| persona_telefono | id BIGSERIAL, id_persona BIGINT, telefono VARCHAR(10) | PK; FK CASCADE; teléfono/ID

persona NOT NULL; regex diez dígitos; UNIQUE (id_persona,telefono) |

No existe unicidad global de correo/teléfono. Un índice de búsqueda no equivale a una restricción UNIQUE. El correo se normaliza en el servicio, no mediante un trigger SQL. JPA ordena ambas colecciones por id ASC. Al sincronizar se eliminan valores ausentes y se agregan nuevos; no se reordenan todos los IDs según el nuevo payload. El correo principal del header es el primero por ese orden persistente.

persona_contacto_emergencia

| Columna | Tipo | Restricción |

| --- | --- | --- |

| id | BIGSERIAL | PK |

| id_persona | BIGINT | NOT NULL; FK a persona, ON DELETE CASCADE |

| id_contacto | BIGINT | NOT NULL; FK a persona, ON DELETE CASCADE |

| id_parentesco | BIGINT | NOT NULL; FK a catálogo de parentesco |

uq_pce_par impide duplicar (id_persona,id_contacto). ck_pce_distintos impide id_persona=id_contacto. SQL no obliga a que id_persona tenga un perfil titular; esa precondición corresponde al servicio. Los ManyToOne de la relación son LAZY y no hacen cascada REMOVE sobre contacto/parentesco.

Catálogos y usuario

| Tabla | Columnas relevantes | Restricciones |

| --- | --- | --- |

| catalogo_ocupacion | id BIGSERIAL, nombre VARCHAR(100) | PK; nombre UNIQUE NOT NULL |

| catalogo_parentesco | id BIGSERIAL, nombre VARCHAR(50) | PK; nombre UNIQUE NOT NULL |

| usuario | id BIGSERIAL, username VARCHAR(50), password VARCHAR(255), rol VARCHAR(20), id_persona BIGINT |

PK; username UNIQUE NOT NULL; password NOT NULL; rol NOT NULL default ROLE_ADMIN; persona UNIQUE NOT NULL,

FK CASCADE |

Usuario.idPersona es un Long, no una relación JPA. La cuenta obtiene su persona mediante un repositorio separado. El modelo no declara longitud 50 para username y su validación permite 255, por lo que Hibernate/SQL pueden divergir.

El script incluye ALTER TABLE usuario ADD COLUMN IF NOT EXISTS id_persona ... NOT NULL UNIQUE REFERENCES persona(id). No rellena datos antiguos; una tabla ya poblada sin esa columna puede rechazar el ALTER. No debe interpretarse como migración general de cuentas.

Índices declarados

| Nombre | Expresión | Uso |

| --- | --- | --- |

| idx_pce_persona | persona_contacto_emergencia(id_persona) | Relaciones salientes |

| idx_pce_contacto | persona_contacto_emergencia(id_contacto) | Compartición y búsqueda de referencias |

| idx_persona_correo_busqueda | persona_correo(lower(trim(correo))) | Coincidencias normalizadas |

| idx_persona_telefono_busqueda | persona_telefono(telefono) | Coincidencias por teléfono |

PKs y UNIQUE producen sus propios índices. El esquema no declara índice dedicado sobre direccion_titular(id_persona) ni perfil_titular(fecha_baja); no se ha medido un plan EXPLAIN en esta revisión.

Semilla y secuencias

data.sql contiene 7 ocupaciones, 11 parentescos, 6 personas, 2 perfiles titulares, 2 direcciones principales, comunicaciones, 4 relaciones de emergencia y 2 cuentas. No se reproducen aquí nombres, comunicaciones, credenciales ni hashes de esas filas.

Las direcciones iniciales tienen calle/número/municipio; otros campos quedan vacíos para completar en edición. La UI actual es más estricta que el servicio y exigirá completar CP, estado y colonia.

Varios INSERT usan ON CONFLICT, pero las direcciones no tienen una clave de conflicto declarada para el par persona/orden. Reejecutar la semilla puede duplicarlas. El script debe ejecutarse una vez sobre una base nueva.

Se insertan IDs explícitos de persona, catálogos y cuentas. El script solo ejecuta setval sobre persona_id_seq; las secuencias de catálogos/usuario no quedan necesariamente alineadas. El alta implícita de una ocupación puede chocar con un ID sembrado. Ver hallazgo R03 (REVISION_TECNICA.md).

DDL automático y alcance de verificación

La configuración versionada utiliza spring.jpa.hibernate.ddl-auto=update. Hibernate puede crear/ajustar estructuras, pero no es evidencia de que aplique los CHECK, índices y longitudes de schema.sql de forma idéntica. Por ejemplo, PersonaTelefono declara longitud 30 y SQL longitud 10.

La revisión contrastó código y scripts; no ejecutó recreación ni consultas de inspección sobre la base de desarrollo. Las pruebas de integración están condicionadas a una base local desechable y quedaron omitidas. No se afirma que la base instalada coincida con este diccionario.

docs/OPERACION_Y_PRUEBAS.md

Instalación, operación y verificación

Requisitos y versiones

- JDK 21 con java accesible desde la terminal.

- Node.js y npm compatibles con Angular CLI 22; package.json no fija engines ni hay .nvmrc. Usar el lockfile para resolver dependencias.

- PostgreSQL con una base de desarrollo vacía y permisos de creación de tablas/índices/secuencias. No hay Docker Compose versionado.

- No se requiere Chrome para las pruebas unitarias: el runner es Vitest con jsdom.

- Python con reportlab y pypdf si se necesita regenerar el PDF. Estas dependencias son del generador documental, no de la API.

Los comandos siguientes se ejecutan manualmente. La revisión documental no recrea, vacía ni modifica bases instaladas.

1. Crear el esquema de desarrollo

Crear una base nueva con la herramienta habitual de PostgreSQL y ejecutar, en este orden:

```
psql -X -v ON_ERROR_STOP=1 -d <base_nueva> -f Backend/demo/src/main/resources/schema.sql
psql -X -v ON_ERROR_STOP=1 -d <base_nueva> -f Backend/demo/src/main/resources/data.sql
```

<base_nueva> es un marcador que debe sustituirse. No pegar estos comandos sin ajustarlo. Configurar autenticación de PostgreSQL fuera de argumentos registrados o documentación. Los SQL son manuales; la configuración actual no garantiza ejecutarlos automáticamente al arrancar Spring.

CREATE TABLE IF NOT EXISTS no elimina columnas existentes. Para obtener la estructura actual sin direccion/ciudad, usar una base vacía. No aplicar data.sql varias veces: sus direcciones no son idempotentes. La reconstrucción borra datos si previamente se decide eliminar la base; esa acción no forma parte de los comandos anteriores.

2. Configurar y arrancar el backend

Ubicación: Backend/demo/src/main/resources/application.properties. Se revisaron nombres de propiedades y opciones no sensibles; se omitieron valores de conexión y firma.

```
| Propiedad | Estado comprobado |
| --- | --- |
| spring.datasource.url | Valor literal configurado; debe apuntar a la base elegida |
| spring.datasource.username, .password | Valores literales; no deben copiarse ni versionarse al cambiar
entorno |
| spring.datasource.driver-class-name | Driver configurado |
| jwt.secret | Valor literal; firma HS256, debe mantenerse fuera de entregables compartidos |
| spring.jpa.hibernate.ddl-auto | update |
| spring.jpa.show-sql | true |
| spring.jpa.properties.hibernate.dialect | org.hibernate.dialect.PostgreSQLDialect |
```

El archivo actual no utiliza placeholders para esas credenciales. No asumir que DATABASE_URL o JWT_SECRET se consumen automáticamente: deben vincularse mediante configuración Spring/overrides y verificarse. El generador PDF no incluye los valores.

```
# Desde Backend/demo
.\mvnw.cmd spring-boot:run
```

Linux/macOS: ./mvnw spring-boot:run. El puerto esperado es 8080. La URL CORS local permitida es http://localhost:4200. No hay endpoint health propio ni Spring Actuator en el POM.

3. Instalar y arrancar el frontend

```
# Desde Frontend
npm.cmd ci
npm.cmd start
```

En otras shells usar npm ci/npm start. En PowerShell, .cmd evita depender de la política de ejecución de npm.ps1. El servidor usa configuración development y puerto 4200 salvo override explícito.

La API se referencia en AuthService, PersonasService y jwtInterceptor. El servicio de CP está en CodigoPostalService. Cambiar solo un archivo de URL no configura completamente otro entorno. Cambiar las entradas styles de angular.json requiere reiniciar ng serve; la entrada vigente es src/styles.scss, que usa Sass para cargar el tema Material.

4. Compilación

```
npm.cmd run build
```

Salida: Frontend/dist/formulario-registro/browser. La configuración production usa hashing. Presupuestos declarados:

Presupuesto	Warning	Error
initial	500 kB	1100 kB
anyComponentStyle	2 kB	4 kB

La configuración fija umbrales, pero el tamaño depende del estado de las fuentes y dependencias. Durante la migración Angular 22 (2026-10-28) se registró build aprobado con avisos de presupuesto; no se repitió tras los cambios visuales posteriores.

5. Pruebas frontend

```
npm.cmd test -- --watch=false
```

La suite usa Vitest, jsdom, TestBed y HttpTestingController en componentes standalone. Durante la migración Angular 22 (2026-10-28) se registraron 92 pruebas aprobadas, salida 0. Es un resultado histórico, no repetido después de los cambios visuales posteriores.

No hay script E2E, configuración Playwright/Cypress ni prueba de todos los flujos conectados a una base real. Una prueba de componentes no verifica el contrato efectivo Spring/Jackson/PostgreSQL. Existe una verificación visual previa del login en Chrome; no equivale a cobertura visual de todas las resoluciones/pantallas.

6. Pruebas backend

```
# Desde Backend/demo
.\mvnw.cmd test
```

En un entorno que resuelve erróneamente la caché como C:\.m2\repository, puede indicarse la carpeta Maven existente:

```
.\mvnw.cmd "-Dmaven.repo.local=<ruta_de_cache_maven>" test
```

No cambiar variables de entorno de usuario/sistema como HOME para corregir este problema.

Clase	Resultado de la revisión
JwtUtilTest	1 aprobada
CuentaServiceTest	7 aprobadas
PersonaContratosTest	5 aprobadas
RegistroCompletoTest	30 aprobadas
DemoApplicationTests	13 excluidas expresamente; sin integración
Total Surefire de la selección unitaria	43 reportadas/ejecutadas, 0 fallos/errores; integración no incluida

Para reproducir la selección unitaria, añadir "-Dtest=JwtUtilTest,CuentaServiceTest,PersonaContratosTest,RegistroCompletoTest" a mvnw.cmd test. Esta selección no inicia DemoApplicationTests aunque haya variables TEST_DATABASE_* presentes.

Las trazas ERROR de casos negativos de contactos aparecen durante la suite por el logger del servicio; no indican por sí mismas una prueba fallida. Hay warnings de deprecación/adjunción de agente Mockito; la ejecución terminó en BUILD SUCCESS.

Integración PostgreSQL, solo sobre una base desechable

DemoApplicationTests requiere TEST_DATABASE_URL que coincida con:

```
jdbc:postgresql://(localhost|127.0.0.1):<puerto>/nomina_backend_test
```

Variables adicionales: TEST_DATABASE_USERNAME (default postgres) y TEST_DATABASE_PASSWORD (default vacío). La condición JUnit comprueba URL local/nombre, no demuestra que la base sea desechable.

```
$env:TEST_DATABASE_URL = 'jdbc:postgresql://127.0.0.1:55439/nomina_backend_test'
# Configurar TEST_DATABASE_USERNAME y TEST_DATABASE_PASSWORD en la sesión según la instancia.
.\mvnw.cmd test
```

Antes de ejecutar, confirmar que es una instancia/base independiente de desarrollo. @BeforeEach crea el esquema y ejecuta TRUNCATE ... RESTART IDENTITY CASCADE; estas pruebas destruyen los datos de la base indicada. Sustituyen ddl-auto por none y spring.sql.init.mode por never; no usan data.sql como preparación.

7. Contraseñas y documentación

La utilidad scripts/HashPassword.java solicita contraseña y confirmación mediante Console, exige al menos 12 caracteres y máximo 72 bytes UTF-8 y genera BCrypt con coste 12. No recibe secretos mediante argumentos. La edición de cuenta de la API usa mínimo 8 y el encoder Spring por defecto; no son políticas idénticas.

```
.\mvnw.cmd dependency:build-classpath -DincludeScope=runtime "-Dmdep.outputFile=target/runtime-
classpath.txt"
$runtimeClasspath = (Get-Content target/runtime-classpath.txt -Raw).Trim()
java --class-path $runtimeClasspath scripts/HashPassword.java
```

No registrar hashes ni claves resultantes en commits o documentos. Para la guía:

```
python ../../docs/back/generar_guia_backend.py --date 2026-10-29
```

Ejecutar desde Backend/demo. El generador y sus entregables viven en docs/back/ de la raíz. Produce el PDF y el manifiesto; no modifica README ni reproduce cuerpos de fuentes que puedan contener secretos. Las fuentes de documentación son Markdown y las referencias de líneas se calculan nuevamente.

8. Diagnóstico

```
| Síntoma | Comprobación concreta |
| --- | --- |
| Login vuelve ante una petición | Examinar estado 401/0/5xx y timeout; el cierre es deliberado |
| Backend cae pero la pantalla permanece | No hay polling; un resultado en caché no realiza HTTP |
| Campos Material sin estilos | Ver entrada styles.scss, import Sass y reiniciar servidor tras cambio de
angular.json |
| Fuentes distintas en otra máquina | Comprobar Google Fonts y conexión; se usan familias de respaldo si
falla |
| Guardado con 404 pero registro existe | Controller PUT puede traducir una RuntimeException interna a 404 |
| Username largo falla aunque formulario lo acepta | SQL limita 50; validación frontend/backend permite 255
|
| Nueva ocupación falla con PK | Revisar secuencia del catálogo tras semilla con IDs explícitos |
| No se ven cambios de otro cliente | Caché en memoria sin TTL ni invalidación remota; recargar |
```

| Vitest no encuentra el DOM | Confirmar jsdom, polyfills zone.js/testing y zone.js/plugins/vitest-patch del target testing |

Alcance de los resultados

Historial de verificación: backend unitario, 43 pruebas seleccionadas aprobadas el 2026-10-15; la integración PostgreSQL se omitió. Frontend: durante la migración Angular 22, build y 92 pruebas Vitest aprobadas el 2026-10-28. Estos resultados no certifican los cambios posteriores ni sustituyen una nueva ejecución. No se registran aquí pruebas de carga, pentest, despliegue ni E2E.

docs/REVISION_TECNICA.md

Revisión técnica y limitaciones conocidas

Revisión base del 15 de octubre de 2026 sobre el código versionado entonces. Los hallazgos proceden de lectura estática, no de explotación, medición de carga ni inspección de una base instalada. La migración posterior actualizó Angular, el builder, el runner de pruebas y las animaciones del frontend; este informe conserva sus resultados y hallazgos con la fecha original, sin certificar el estado de producción.

Resultados comprobados en la revisión base (2026-10-15)

- npm.cmd run build: aprobado; conserva warnings de presupuesto inicial/CSS.

- Frontend en revisión base: Angular/Karma/ChromeHeadless, 94 pruebas aprobadas. Este runner fue sustituido por Vitest durante la migración del 2026-10-28; no tomar este número como verificación actual.

- Maven: 43 unitarias seleccionadas, ejecutadas y aprobadas; 13 integraciones excluidas expresamente.

- Login: verificación visual previa con tema Material activo, controles de 56 px y botón de 49 px. No hay auditoría visual completa.

- No se usó ni vació la base de desarrollo para esta revisión.

Hallazgos

ID	Prioridad de revisión	Evidencia	Implicación
R01	Alta	CuentaService.actualizarLogin, CuentaComponent y schema.sql	Se aceptan username de hasta 255, pero SQL permite 50 y JPA no fija 50. Fallos de persistencia pueden traducirse a 409 engañoso
R02	Alta	SecurityConfig, JwtFilter, AuthController	Seguridad exige autenticación, no rol/pertenencia; JWT crea autoridades vacías, no reconsulta cuenta/baja. No hay autorización por titular ni revocación server-side
R03	Alta	data.sql y creación implícita de ocupaciones	Solo se reajusta persona_id_seq; catálogos/cuentas con IDs explícitos conservan secuencias potencialmente atrasadas
R04	Alta	application.properties	Valores de conexión y clave JWT literales en una fuente versionada. Se omitieron en los documentos; externalización/rotación debe tratarse separadamente
R05	Media	PersonaService.reactivar frente a bloquearContactos	Altas usan advisory lock global; reactivaciones no. Altas/reactivaciones concurrentes pueden superar el límite lógico de 20; SQL no lo garantiza
R06	Media	PersonaService.validarTitular, entidades y SQL	Género del titular no se valida explícitamente y los contactos no tienen control de longitud 50. Fechas futuras no se rechazan; validaciones de la UI no equivalen a seguridad del servidor
R07	Media	PersonaController	PUT/DELETE traducen RuntimeException a 404; POST a 400; otros endpoints delegan en Spring. No hay contrato homogéneo de errores ni distinción estable entre fallos internos/cliente
R08	Media	PersonasService.consultarPersonasCache, consultarCompartido, obtenerContactos	Eliminar entrada no cancela peticiones antiguas. Algunos finalizers/error handlers borran por URL/ID sin comprobar identidad; pueden afectar una petición nueva tras invalidación. No es persistencia de sesión en disco
R09	Media	OcupacionController y PersonasService.obtenerOcupaciones	API devuelve objetos {id,nombre}, pero servicio TypeScript declara string[]; Registro usa any[] y fallback o.nombre || o, ocultando el desacuerdo
R10	Media	data.sql, direccion_titular	Reejecutar semilla puede duplicar direcciones. No hay UNIQUE persona/orden; la coherencia del orden depende del servicio
R11	Media	Métodos JPA de listado y convertirAResumen	Se reduce JSON, pero se recuperan entidades y asociaciones para construirlo. No se midió N+1 ni EXPLAIN; no debe afirmarse que el listado ejecuta solo una

proyección SQL reducida |

| R12 | Media | AuthController.login, JwtFilter | Login distingue 404/401 y no valida body completo; filtro acepta identidad de firma válida sin comprobar que usuario siga existiendo. Cambio de username no revoca token criptográficamente |

| R13 | Media | Logs de PersonasService y ContactosComponent | Se registra el objeto de error HTTP/stack completo. Puede incluir body o detalles internos; no hay sanitización de logs demostrada |

| R14 | Baja | PersonaTelefono, schema.sql, ddl-auto=update | Longitud JPA 30 y SQL 10; DDL Hibernate y SQL manual pueden producir esquemas distintos |

| R15 | Baja | schema.sql ALTER de usuario | ADD COLUMN NOT NULL no rellena una tabla antigua; no constituye migración válida de datos existentes |

| R16 | Operativa | jwtInterceptor y cachés | Salida por backend caído se dispara en la siguiente petición. No hay health polling; pérdida de red/CORS también puede producir estado 0 y cerrar sesión |

| R17 | Operativa | PersonaResumenDTO | Solo municipio principal en listado. La búsqueda usa los datos disponibles; no busca municipios de direcciones secundarias no descargadas |

| R18 | Operativa | AuthService y almacenamiento | JWT en localStorage, borrador en sessionStorage; no refresh token. Validar exp en cliente no valida firma. El intervalo de 250 ms observa almacenamiento, no backend |

Las prioridades indican impacto potencial para futuras tareas; no son una certificación de vulnerabilidad ni una autorización para cambiar reglas funcionales en esta entrega.

Diferencias entre requisitos de captura y backend

| Concepto | UI | Servicio Java/SQL |

| --- | --- | --- |

| Nombres titular | Mínimo 2 en controles | Servicio exige no blank y máximo 100; controles no imponen el mismo máximo |

| Dirección | País/estado/municipio/colonia/CP/calle/número obligatorios | Servicio admite vacíos excepto calle/municipio; CP vacío permitido por SQL |

| Aceptación de términos | Obligatoria en alta; deshabilitada en edición | No se envía ni persiste en PersonaDTO |

| Ocupaciones | Select con catálogo | Servicio crea ocupación desconocida; parentesco debe existir |

| Contraseña nueva | Cuenta: mínimo 8, máximo 72 bytes | Utilidad administrativa de hash: mínimo 12 y coste BCrypt 12 |

| Relaciones de contacto | Confirmación y mínimo 1 | Servicio valida mínimo; SQL permite cero y no exige perfil en origen |

Cobertura y tareas pendientes de validación

La suite usa mocks y pruebas HTTP simuladas; las pruebas de integración no se ejecutaron. Quedan sin comprobación empírica: serialización real de errores de Spring Security, URLs fuera de localhost, CORS del proveedor postal, persistencia y rollback con PostgreSQL, concurrencia de reactivaciones, planes de consulta, flujo completo en navegador con backend real y revisión de accesibilidad.

La actualización selectiva del listado se verificó con respuestas HTTP simuladas: reemplazo por ID, conservación de otras filas/detalles, ediciones sucesivas, petición inicial tardía y escritura de una sesión anterior. El encabezado se comprobó con cambios propios/ajenos, campos no visibles, contacto compartido, guardado fallido y perfil antiguo pendiente.

Los números de pruebas se documentan con fecha y no deben copiarse indefinidamente. Los manifiestos de la guía tienen hashes/line numbers para identificar la revisión exacta; después de cambiar fuentes, regenerarlos.

Frontend/README.md

Frontend Angular de administración de personas

SPA standalone con Angular/Material/CDK 22.x, TypeScript 6.x y RxJS 7.8.x. Las decisiones y el historial están en Detalle de migración Angular 22 (../razi/Detalle_Migracion_Angular_22.docx). Arquitectura (../docs/ARQUITECTURA.md), API (../docs/API.md), persistencia (../docs/BASE_DE_DATOS.md), operación/pruebas (../docs/OPERACION_Y_PRUEBAS.md) y revisión (../docs/REVISION_TECNICA.md) son la documentación compartida.

Ejecución

Desde esta carpeta:

```
npm.cmd ci
npm.cmd start
npm.cmd run build
npm.cmd test -- --watch=false
```

Servidor: http://localhost:4200; backend esperado: http://localhost:8080. En otras shells usar npm sin .cmd. Build production genera dist/formulario-registro/browser; watch usa development. No hay environment central de URLs, SSR, Service Worker, Docker ni E2E versionado.

Entradas y organización

Archivo/carpeta	Responsabilidad
src/main.ts	Bootstrap standalone
src/app/app.config.ts	Router, detección Zone, HttpClient con XHR e interceptor JWT
src/app/app.routes.ts	Rutas, guards, títulos y modos de contactos
src/app/app.component.ts	Redirección global ante pérdida de token
components/	Pantallas, layout y feedback
services/	Transporte HTTP, sesión, cachés y borrador
interfaces/	Contratos TypeScript; no validación JSON en runtime
guards/	Autenticado/invitado
interceptors/	Bearer, timeout y fallos de la API
src/styles.scss	Variables y reglas globales; @use del tema Material
src/material-typography.scss	Material 3 Azure/Blue y familias tipográficas

Login usa ngModel; registro/contactos/cuenta, formularios reactivos. Señales gestionan datos visibles, carga, selección y errores; RxJS gestiona solicitudes y coincidencias. No hay Store NgRx ni un patrón único de teardown en todas las suscripciones.

Rutas

Ruta	Componente	Comportamiento
/	Redirect	/inicio
/inicio	InicioComponent + guestGuard	Login; sesión válida va a dashboard
/dashboard	DashboardComponent	Agregado, capacidad y top 5 ocupaciones
/personas	PersonasComponent	Resumen, búsqueda local, detalles y baja
/personas/desactivadas	PersonasDesactivadasComponent	Búsqueda local y reactivación
/registro	RegistroComponent	Alta, primer paso; borrador, sin POST
/registro/:id	RegistroComponent	Editar titular activo por ID
/contactos	ContactosComponent	Segundo paso de alta; exige borrador
/contactos/:id	ContactosListaComponent	Relaciones; quitar manteniendo mínimo uno
/contactos/:id/editar/:contactoId	ContactosComponent	Editar relación/persona compartida

| /contactos/:id/agregar | ContactosComponent | Agregar conservando relaciones previas |

| /cuenta | CuentaComponent | Credenciales; logout al guardar |

| Otras | Redirect | /inicio |

Administrativas dentro de AdminLayout, con canActivate y canActivateChild. El guard valida expiración local, no firma; el backend es autoridad final. No hay returnUrl. Sidebar calcula subapartados por URL de edición/persona/contacto; breakpoint del menú 760 px.

Contratos y formularios

PersonaResumen usa direcciones?:Pick<Direccion,'municipio'>[]; el backend devuelve solo municipio principal. PersonaDetalle contiene género, fecha y direcciones completas. Usuario representa el formulario y usa Direccion[]. Detalles usa /{id}/detalles; editar usa /{id}. Contactos se descargan por ruta separada. No existen campos JSON direccion/ciudad.

La búsqueda local normaliza NFD, elimina diacríticos y usa datos descargados; no consulta SQL ni municipios secundarios. Cerrar/reabrir detalles o navegar conserva datos en PersonasService.

Registro usa FormArray para direcciones/correos/teléfonos y mantiene mínimo uno. cp se convierte a codigoPostal; nombres/apellidos separados a nombre/apellido. El alta guarda registro_borrador en memoria/sessionStorage y navega a contactos; solo el segundo paso ejecuta POST transaccional. Edición bloqueada hasta cargar persona. Las direcciones se conservan sin consultar CP al abrir; se permite captura manual y se ignoran respuestas postales obsoletas o de grupos eliminados.

Contactos usa comunicaciones variables, debounceTime(350), switchMap y revisiones. Reutilización confirmada bloquea datos personales y envía ID/parentesco. Edición completa de persona compartida exige confirmación. Guardado conserva IDs no editados; quitar sustituye el conjunto completo, no llama DELETE de persona. Nunca se guarda antes de cargar/verificar.

Cuenta valida username, contraseña actual, nueva y confirmación, con máximo de 72 bytes UTF-8. Tras ok:true limpia formulario, cierra sesión y vuelve al login. El máximo 255 de username difiere del SQL 50; ver revisión R01.

Servicios, cachés y errores

| Servicio | Responsabilidad |

| --- | --- |

| PersonasService | Personas/contactos/catálogos; Maps de datos y pendientes; borrador; Subject de

actualización del perfil |

| AuthService | JWT localStorage, BehaviorSubject, perfil en señal, expiración y cambios externos |

| CodigoPostalService | GET https://www.correosmexico.com.mx/api/cp con parámetro cp; sin JWT |

| FeedbackService | Avisos de 5 s, confirmación Promise y señales |

JWT: jwt_token/localStorage. Borrador: registro_borrador/sessionStorage. Header usa datos del login o /auth/me al recargar; actualización selectiva por idPersona cuando cambia nombre completo o correo principal, sin polling de perfil. La fila del PUT permite actualizar la cuenta propia localmente; contactos compartidos solo disparan /auth/me si afectan esos campos de la cuenta activa.

Listas activas/inactivas y detalles: memoria sin TTL. Contactos: 30 s desde crear entrada. Parentescos: sesión. Formulario por ID/ocupaciones/resumen: solo compartir pendientes. Editar un titular conserva el listado y reemplaza solo su ID con el resumen del PUT, sin otro GET. Invalida únicamente los detalles de ese ID; altas/bajas/reactivaciones/contactos y sesión mantienen invalidación completa. Contactos se invalidan tras edición de personas/contactos/sesión. Matriz y límites (../docs/ARQUITECTURA.md).

jwtInterceptor afecta solo http://localhost:8080/api/. Añade Bearer, timeout por emisión 15 s, logout para 401/0/5xx si coincide token capturado. No logout por 403/validación/CP externo. Detecta caída al pedir datos, no periódicamente. PersonasService convierte HttpErrorResponse a Error con mensaje; no siempre se conserva status original en consumidores.

Tema y presentación

Sora: títulos; Plus Jakarta Sans: controles/lectura; IBM Plex Mono: estadísticas/campos numéricos. Familias centralizadas en --app-font-body, --app-font-heading, --app-font-numeric. Google Fonts en index.html con display=swap, preconnect y fallbacks.

El tema incluye core, botones/icon-buttons/FAB, card, checkbox, form-field/input, list, progress-bar/spinner, select, sidenav, table y toolbar. Añadir módulos Material puede requerir extender sus mixins. No reemplazar base/color/densidad por solo tipografía. Entrada Sass en build y pruebas Angular; reiniciar ng serve tras cambios de angular.json. Las pruebas usan Vitest y jsdom.

Feedback usa dialog nativo/showModal, no MatDialog. Login/dashboard tienen animaciones y reduced-motion. Las listas incluyen tarjetas móviles, estados de carga y atributos ARIA. No hay auditoría completa de accesibilidad ni visual automatizada.

Verificación

Durante la migración (2026-10-28) se registraron build de producción aprobado y 92 pruebas Vitest aprobadas, con avisos de presupuesto de tamaño. Ese resultado es histórico y no se repitió tras los cambios visuales posteriores. No hay E2E ni pruebas de todos los componentes conectados a backend real.

Antes de modificar HTTP contrastar DTO Java/TypeScript. La respuesta de ocupaciones es objeto id/nombre aunque el servicio declara string[] y Registro usa any[]; no asumir que los generics equivalen a contrato runtime. Actualizar documentación compartida al cambiar modelo/rutas.

Las funciones de componentes, servicios, guards e interceptor incluyen comentarios breves de propósito y lógica. Se documentan también getters, helpers locales y estado derivado relevante; no se añaden comentarios repetitivos a cada callback HTTP o prueba. Los comentarios no cambian contratos ni comportamiento.

Edición del titular: PATCH /api/personas/{id} intercambia solo campos modificados y sus valores persistidos. El servicio compara con una copia en memoria, omite peticiones sin cambios y fusiona los deltas de lista/detalles sin GET posterior. Las colecciones modificadas se envían completas; PUT completo permanece disponible en la API por compatibilidad. La guía explicada de Angular está en razi (../razi/Guia_Funcionamiento_Angular.docx); el informe por etapas está en Detalle de migración Angular 22 (../razi/Detalle_Migracion_Angular_22.docx).

Backend/demo/README.md

Backend de administración de personas

Java 21, Spring Boot 4.1.1, MVC, Security, Data JPA/JDBC, PostgreSQL y JJWT 0.11.5. Documentación completa: arquitectura (../../docs/ARQUITECTURA.md), API (../../docs/API.md), BD (../../docs/BASE_DE_DATOS.md), operación (../../docs/OPERACION_Y_PRUEBAS.md), hallazgos (../../docs/REVISION_TECNICA.md).

Ejecución

```
.\mvnw.cmd spring-boot:run
.\mvnw.cmd test
```

Wrapper Maven 3.9.16, puerto esperado 8080, CORS http://localhost:4200. El POM fija Java 21 y contiene plugin Asciidoctor en prepare-package; no hay OpenAPI/REST Docs efectivo que sustituya los contratos Markdown. En Linux/macOS usar ./mvnw.

Configuración: src/main/resources/application.properties. Datasource/clave JWT contienen valores literales sensibles; se omiten aquí/PDF. ddl-auto=update, show-sql=true, dialecto PostgreSQL. No asumir consumo automático de DATABASE_URL/JWT_SECRET sin vincularlo mediante propiedades/overrides Spring. Externalización/rotación pendiente registrada.

Mapa de clases

```
| Clase/paquete | Función |
| --- | --- |
| DemoApplication | Bootstrap y escaneo Spring |
| AuthController | Login, perfil y credenciales propias |
| PersonaController | Listados, adicionales, formulario, baja/reactivación, contactos/coincidencias y
agregado |
| OcupacionController / ParentescoController | Lectura de catálogos |
| PersonaService | Validación, transacciones, DTOs, direcciones y relaciones/limpieza |
| CuentaService | Username, BCrypt, perfil del header y credenciales |
| PersonaRepository | JPA, SQL nativo agregado, búsqueda, advisory lock y DELETE de huérfanos |
| UsuarioRepository | Username, protección ROLE_ADMIN y IDs administradores en lote |
| Repositorios de catálogo | CRUD y resolución exacta por ID/nombre |
| JwtUtil / JwtFilter | HS256, una hora, subject y autenticación stateless |
| SecurityConfig | CORS, autenticación, CSRF deshabilitado y BCrypt |
```

DTOs separan PersonaResumenDTO (tabla), PersonaDetalleDTO (adicionales), PersonaDTO (formulario), ContactoDTO y agregados. PUT del titular devuelve PersonaActualizadaDTO (ok y PersonaResumenDTO de la fila editada); otras escrituras devuelven ResultadoDTO; DELETE, 204. El perfil UsuarioActualDTO incluye idPersona para identificar cambios de la cuenta activa.

Modelo y reglas

Titular por PerfilTitular con PK compartida; no es_titular en SQL. Activo si fecha_baja null. Direcciones solo en direccion_titular, orden principal cero; perfil/DTO sin direccion/ciudad.

Alta: advisory lock, validar personales/comunicaciones/contactos/direcciones, contar 20 activos, guardar titular, crear/reutilizar contactos y vincular. Una transacción y confirmación 201; no devuelve ID.

Actualizar: personales/ocupación, reemplazo de direcciones y sincronización de comunicaciones; no sustituye contactos desde PersonaDTO. Ocupación desconocida se crea; parentesco debe existir. Reactivar no adquiere el mismo lock, por lo que el límite no está protegido en toda concurrencia.

Baja: solo activo, prohíbe cuenta ROLE_ADMIN, conserva datos. Reactivación: solo dado de baja, rechaza si ya hay 20 activos. Contactos mínimo uno sin máximo; direcciones entre 1 y 20. Datos del contacto pertenecen a una Persona compartida; parentesco al vínculo.

PUT contactos sustituye conjunto completo; ausentes se desvinculan. Cleanup después de saveAndFlush elimina la persona solo sin perfil/cuenta/vínculos entrantes o salientes. Los vínculos no hacen cascada REMOVE hacia la persona.

Listados son DTO reducido construido desde entidades, no proyección SQL especializada; solo municipio principal. Detalles: género/fecha/direcciones. Formulario no incluye contactos rellenados. Contactos: SQL string_agg y Server-Timing. Dashboard: agregados y top 5. No paginación.

Seguridad y errores

Login/OPTIONS públicos; demás peticiones requieren autenticación. JWT HS256, sub=username, duración 3600000 ms. Filtro no crea autoridades ni revalida rol/baja/existencia de cuenta. No hay refresh/revocación/health endpoint. La caída del backend se maneja en Angular al hacer una petición.

Los controllers no unifican errores: POST/PUT/DELETE tienen mapeos distintos de RuntimeException; algunos endpoints delegan en Spring. No garantizar formato JSON/código para todos los fallos internos sin integración HTTP.

SQL y pruebas

Ejecutar schema.sql (src/main/resources/schema.sql), después data.sql (src/main/resources/data.sql), una vez sobre base vacía. No migraciones activas; IF NOT EXISTS no elimina columnas. Semilla con IDs explícitos, solo persona_id_seq reajustada y direcciones no idempotentes; varios campos iniciales pendientes de completar.

2026-10-15: ejecución seleccionada de 43 pruebas unitarias, todas aprobadas, sin fallos/errores. JwtUtilTest: 1; CuentaServiceTest: 7; PersonaContratosTest: 5; RegistroCompletoTest: 30. Las 13 de DemoApplicationTests se excluyeron expresamente; no se ejecutó integración con PostgreSQL.

Integración requiere TEST_DATABASE_URL local /nomina_backend_test y ejecuta TRUNCATE CASCADE. Nunca usar desarrollo. Variables y procedimiento (../../docs/OPERACION_Y_PRUEBAS.md). Trazas ERROR de casos negativos no equivalen a fallo de suite.

Guía PDF e índice de fuentes

- PDF (../../docs/back/Guia_backend_archivos_y_lineas.pdf): documentación técnica e índice de declaraciones/tablas con líneas calculadas.

- Manifiesto (../../docs/back/referencias_backend.json): rutas, SHA-256, line counts y referencias, sin cuerpos de configuración/semilla.

- Generador (../../docs/back/generar_guia_backend.py): lee Markdown/fuentes, no reescribe README ni publica secretos/datos personales.

```
python ../../docs/back/generar_guia_backend.py --date 2026-10-29
```

Requiere ReportLab/pypdf, usa fuente del sistema si existe y fallback portable. Mermaid se conserva como notación textual en PDF. Regenerar después de modificar fuentes; las líneas son de una revisión, no IDs estables.

scripts/HashPassword.java es una utilidad interactiva BCrypt con coste 12, mínimo 12 caracteres/máximo 72 bytes. Su política difiere de edición de cuenta. Ver procedimiento en Operación; no copiar credenciales/hashes a entregables.

Los métodos declarados de controllers, servicios, repositorios, seguridad, entidades, PersonaDTO y la utilidad BCrypt tienen comentarios breves de propósito. Getters y setters describen el dato expuesto; las operaciones de negocio resumen su validación o efecto. No se modificaron contratos ni lógica al documentarlos.

Edición del titular: PATCH /api/personas/{id} intercambia solo campos modificados y sus valores persistidos. El frontend compara con una copia en memoria, omite peticiones sin cambios y fusiona deltas de lista/detalles sin GET posterior. Las colecciones modificadas se envían completas; PUT completo permanece disponible en la API por compatibilidad.

docs/front/FRONTEND_ESPECIFICACION_AGENTES.md

Contexto técnico frontend

Fuentes: README frontend (../../Frontend/README.md), arquitectura (../ARQUITECTURA.md), API (../API.md), operación (../OPERACION_Y_PRUEBAS.md) y revisión (../REVISION_TECNICA.md).

Stack y ejecución

Angular/Material/CDK 22.x, TypeScript 6.x, RxJS 7.8.x; standalone. npm.cmd start/build/test (scripts de package.json). Build con @angular/build; pruebas Vitest/jsdom mediante el builder unit-test. Estilos globales en src/styles.scss con Sass y @use Material. API local http://localhost:8080/api/, URLs en servicios/interceptor, CP externo.

Invariantes

- Listas resumidas y municipio principal; detalles /{id}/detalles; editar /{id} completo. No reintroducir direccion/ciudad ni descargar contactos en el formulario.

- PersonaResumen tiene dirección parcial; Usuario completa. Comunicaciones variables, mínimo una de cada tipo; direcciones entre 1 y 20 en backend; contactos mínimo uno sin máximo.

- Alta en dos pasos: borrador memoria/sessionStorage y POST final transaccional con contactos. Edición titular no sustituye contactos.

- Convertir cp a codigoPostal y nombres separados a nombre/apellido. Conservar direcciones al abrir y permitir captura manual postal.

- Contactos reutilizados: confirmar, bloquear personales, enviar ID/parentesco. Editar compartidos completos afecta otros titulares. PUT reemplaza lista; conservar demás IDs.

- Guards validan expiración local, no firma. guestGuard dirige autenticados a dashboard. Preservar navegación contextual y bloqueos/carga.

Estado y errores

- JWT localStorage; borrador sessionStorage. Logout limpia borrador/perfil/cachés.

- Header desde login o /auth/me con idPersona. Actualizarlo solo si cambia nombre completo/correo principal de esa persona; usar los campos persistidos del PATCH sin GET adicional. En contactos compartidos consultar /auth/me solo si esos campos afectan a la cuenta activa.

- Listas/detalles sin TTL; al editar un titular reemplazar su fila por ID sin descargar toda la lista y fusionar sus detalles modificados. Otras escrituras/sesión conservan invalidación completa. Contactos 30 s; parentescos por sesión; formulario/resumen/ocupaciones solo pendientes.

- API local: timeout 15 s y logout ante 401/0/5xx con mismo token capturado. No logout por 403/validación/CP externo. Detección de caída al pedir, no periódica.

- No persistir cachés personales en almacenamiento del navegador.

UI y verificación

Tema Material completo para módulos utilizados: core/base/color/densidad/tipografía; extender al agregar otro módulo. Sora títulos, Plus Jakarta Sans lectura, IBM Plex Mono cifras; variables --app-font-*. Mantener reduced-motion, feedback, ARIA y vistas móviles.

Migración 2026-10-28: se registraron build aprobado y 92 pruebas Vitest aprobadas; resultado histórico, no repetido tras los cambios visuales posteriores. No E2E/integración HTTP real. Consultar hallazgos antes de afirmar igualdad de validaciones/tipos/cachés. Sincronizar DTOs, interfaz y documentación al modificar contratos.

Edición del titular: PATCH /api/personas/{id} intercambia solo campos modificados y sus valores persistidos. El servicio compara con una copia en memoria, omite peticiones sin cambios y fusiona cambios de lista/detalles sin GET posterior. Las colecciones modificadas se envían completas; PUT completo permanece disponible por compatibilidad.

docs/back/BACKEND_ESPECIFICACION_AGENTES.md

Contexto técnico backend

Fuentes: README backend (../../Backend/demo/README.md), API (../API.md), BD (../BASE_DE_DATOS.md), arquitectura (../ARQUITECTURA.md), operación (../OPERACION_Y_PRUEBAS.md), revisión (../REVISION_TECNICA.md).

Stack/capas

Java 21, Spring Boot 4.1.1, MVC/Security/JPA/JDBC, JJWT 0.11.5 y PostgreSQL. Wrapper Maven 3.9.16. Controller -> servicio transaccional -> repositorio; DTOs de personas, entidades de catálogos. Configuración con valores literales sensibles: no copiarlos.

Modelo y contratos

- Titular por PerfilTitular/PK compartida; no es_titular. Activo con fecha_baja null.

- Direcciones exclusivamente direccion_titular, orden principal cero; perfil/DTO sin direccion/ciudad. Esquema/semilla de base nueva, sin migraciones activas.

- Comunicaciones variables, mínimo una de cada tipo; contactos mínimo uno sin máximo; direcciones entre 1 y 20.

- Parentesco del vínculo, persona compartida. No REMOVE desde vínculo hacia contacto.

- Omitidos en sustitución se desvinculan; cleanup solo sin perfil/cuenta/vínculos entrantes o salientes.

- ADMIN vinculado no se da de baja; reactivar solo dado de baja con capacidad disponible.

- Listado PersonaResumenDTO, adicionales PersonaDetalleDTO, editar PersonaDTO sin contactos rellenados. PUT titular devuelve PersonaActualizadaDTO con su fila de listado; otras escrituras ResultadoDTO; DELETE 204. UsuarioActualDTO incluye idPersona.

- POST crea con contactos; PUT titular no sustituye contactos; PUT contactos sustituye conjunto entero.

- El servicio crea ocupación desconocida; parentesco debe existir. No documentar la regla contraria.

- Advisory lock en alta/sustitución contactos, no reactivación; límite no garantizado en toda concurrencia.

- Filtro verifica JWT, no roles/existencia/baja/pertenencia. Solo login/OPTIONS públicos.

- No formato uniforme de errores/ControllerAdvice. Consultar discrepancias UI/servicio/SQL al cambiar validación.

Pruebas/documentación

mvnw.cmd test desde demo. Integración requiere TEST_DATABASE_URL local /nomina_backend_test; usa TRUNCATE CASCADE. Nunca desarrollo. Revisión 2026-10-15: 43 unitarias aprobadas y 13 integraciones excluidas expresamente de la ejecución.

Actualizar API/BD/READMEs al cambiar modelo/rutas. PDF/manifiesto con generador que calcula referencias, no reescribe README ni reproduce secretos o datos personales. Regenerar tras cambios; no conservar referencias a archivos eliminados. El frontend usa Angular 22 y Vitest; consulta el README frontend y el informe de migración en razi.

Edicion del formulario: PATCH /api/personas/{id} intercambia solo campos modificados y sus valores persistidos. Frontend compara con una copia en memoria, omite peticiones sin cambios y fusiona deltas de lista/detalles sin GET posterior. Colecciones modificadas se envian completas; PUT completo permanece compatible.

Indice de archivos y declaraciones

Backend/demo/.gitattributes

3 lineas; SHA-256: 1c8cc8fb65937e0ee87a7ed50745c6f87fb78ded993fb796aa800c877cc6d517

Backend/demo/.gitignore

33 lineas; SHA-256: 089382e4d7944ab6cb6900a1269678ebff93c00d441ada788cd05a8da182b22b

Backend/demo/.mvn/wrapper/maven-wrapper.properties

3 lineas; SHA-256: 488e1b3f2e641779d4636abf9390845f901e64607261bc3c0b0bfe4fe96e6706

Backend/demo/AGENTS.md

12 lineas; SHA-256: 96e8f0b3ef868457c52cfbcd80a7a32dbf1f7fb229e7a8fe4c801bb86cf316da

L1: seccion Contexto de agentes — Backend

L5: seccion Reglas rápidas

Backend/demo/README.md

78 lineas; SHA-256: 02c01a828abe053162f61fbbfb7b3421758a48f83c217b310ba5ea3c79321cb8

L1: seccion Backend de administración de personas

L5: seccion Ejecución

L16: seccion Mapa de clases

L34: seccion Modelo y reglas

L48: seccion Seguridad y errores

L54: seccion SQL y pruebas

L62: seccion Guía PDF e índice de fuentes

Backend/demo/mvnw

295 lineas; SHA-256: cae96cef89ebea3531221f4ae17c23cf8edf67d00eae8306d4186ae1bbed4d02

Backend/demo/mvnw.cmd

189 lineas; SHA-256: 46eedb8419bd14fe70d5bb2916d7b6f51806e51b39d5b76a42610384ca929c1c

Backend/demo/pom.xml

137 lineas; SHA-256: 08d655df6a9f0862c150f8b0007b89281bdd1e619e5a3e9954d937313f7b35fb

Backend/demo/scripts/HashPassword.java

31 lineas; SHA-256: 1599fa458ab3945867a199860e684309fe9234946844db5e89981bab89ee5a70

L8: tipo HashPassword

L10: metodo main

Backend/demo/src/main/java/com/example/demo/DemoApplication.java

14 lineas; SHA-256: e1bd99f8bd2cbf0f10991838de37dbc13063f1057834bc1095efe82d0bc64dbb

L7: tipo DemoApplication

L10: metodo main

Backend/demo/src/main/java/com/example/demo/config/SecurityConfig.java

55 lineas; SHA-256: b70476be6e1e267a7b606f17dfe9bd71017445465f9a16f5fa2c67ef571ed958

L18: tipo SecurityConfig

L24: metodo securityFilterChain

L44: metodo userDetailsService

L52: metodo passwordEncoder

Backend/demo/src/main/java/com/example/demo/controller/AuthController.java

81 lineas; SHA-256: 61911103ee4d5325f3a9df429fe92f8aebccf94ed932ab4c3706477a775dbf3b

L19: tipo AuthController

L35: metodo obtenerUsuarioActual

L41: metodo obtenerDatosLogin

L47: metodo actualizarLogin

L55: metodo login

Backend/demo/src/main/java/com/example/demo/controller/OcupacionController.java

21 lineas; SHA-256: cc811cdd8c1f93314f224d5a2eb61f63833b9d354821d06b72d7e293292bbbef

L11: tipo OcupacionController

L18: metodo obtenerOcupaciones

Backend/demo/src/main/java/com/example/demo/controller/ParentescoController.java

24 lineas; SHA-256: 3a3e7f5667639715fa1c0ea0f808d39ca57d2f854a1bad60426823bbbd725f2f

L11: tipo ParentescoController

L21: metodo obtenerTodos

Backend/demo/src/main/java/com/example/demo/controller/PersonaController.java

152 lineas; SHA-256: 2a7393963ea00bec6abf317a60f2f2a66485630e454c2932a0349af71422a8fe

L21: tipo PersonaController

L24: metodo buscarContactos

L31: metodo obtenerContactos

L42: metodo guardarContactos

L53: metodo obtenerResumen

L59: metodo obtenerInactivos

L65: metodo obtenerDetalle

L71: metodo obtenerPorId

L77: metodo reactivar

L88: metodo obtenerPersonas

L103: metodo guardarPersona

L118: metodo actualizarPersona

L133: metodo actualizarParcial

L140: metodo eliminarPersona

Backend/demo/src/main/java/com/example/demo/dto/BusquedaContactoDTO.java

3 lineas; SHA-256: 16adf76b407c7368ccc6dc06cbf975548b86fd16f1e34199e9db637bf6a296a2

L3: tipo BusquedaContactoDTO

L3: metodo BusquedaContactoDTO

Backend/demo/src/main/java/com/example/demo/dto/ContactoDTO.java

8 lineas; SHA-256: e11e38730abb827200e60364da4b8625c774f8950b7b9db95d2d6d98792d0a68

L6: tipo ContactoDTO

L6: metodo ContactoDTO

Backend/demo/src/main/java/com/example/demo/dto/EditarCuentaDTO.java

3 lineas; SHA-256: 3c98f2ea42fff341955cb53db9aabf1aa5d086fe043802a31f86d1ead5c94b4d

L3: tipo EditarCuentaDTO

L3: metodo EditarCuentaDTO

Backend/demo/src/main/java/com/example/demo/dto/PersonaActualizadaDTO.java

4 lineas; SHA-256: 6def8e1f98c64d7534bdd5b3eac2d21c9db2a95fbbf90475015e8d567878e012

L4: tipo PersonaActualizadaDTO

L4: metodo PersonaActualizadaDTO

Backend/demo/src/main/java/com/example/demo/dto/PersonaDTO.java

71 lineas; SHA-256: 828fd2ddd7b3cdcf0d3a711bc6c92bbe88e0b71d5872c3f969ab49f3ac836ba5

L5: tipo PersonaDTO

L13: metodo getCorreos

L15: metodo setCorreos

L17: metodo getTelefonos

L19: metodo setTelefonos

L21: metodo getDirecciones

L23: metodo setDirecciones

L25: metodo getContactosEmergencia

L27: metodo setContactosEmergencia

L40: tipo DireccionDTO

L40: metodo DireccionDTO

L44: metodo getId

L46: metodo setId

L48: metodo getNombre

L50: metodo setNombre

L52: metodo getApellido

L54: metodo setApellido

L56: metodo getFechaNacimiento

L58: metodo setFechaNacimiento

L60: metodo getGenero

L62: metodo setGenero

L64: metodo getOcupacion

L66: metodo setOcupacion

L68: metodo getFechaBaja

L70: metodo setFechaBaja

Backend/demo/src/main/java/com/example/demo/dto/PersonaDetalleDTO.java

7 lineas; SHA-256: 965e5981488b35ea58da2e010b1250df27aaee72d582853feab6fb4667a85547

L6: tipo PersonaDetalleDTO

L6: metodo PersonaDetalleDTO

Backend/demo/src/main/java/com/example/demo/dto/PersonaParcialDTO.java

24 lineas; SHA-256: b2477a0dfd37e90181ec359872f79851b97ffe6fc66bff7f006bab0add2324c4

L4: tipo PersonaParcialDTO

L7: metodo camposPresentes

Backend/demo/src/main/java/com/example/demo/dto/PersonaResumenDTO.java

10 lineas; SHA-256: 6d68d607fb0fc55b11d131ea682761c1146fe6824680d8c5771630b9232767cd

L6: tipo PersonaResumenDTO

L6: metodo PersonaResumenDTO

L9: tipo DireccionResumenDTO

L9: metodo DireccionResumenDTO

Backend/demo/src/main/java/com/example/demo/dto/ResultadoDTO.java

3 lineas; SHA-256: c25001a6773ee0a6d532312d6e4bf6d4fcc7b9c79ed72f45a594cb84accbc8d3

L3: tipo ResultadoDTO

L3: metodo ResultadoDTO

Backend/demo/src/main/java/com/example/demo/dto/ResumenDTO.java

7 lineas; SHA-256: eb75d1dcfb81e399314660fb9b6fe7d54e7550ae3522885927df72c6641a7791

L5: tipo ResumenDTO

L5: metodo ResumenDTO

L6: tipo Ocupacion

L6: metodo Ocupacion

Backend/demo/src/main/java/com/example/demo/dto/UsuarioActualDTO.java

3 lineas; SHA-256: b994e11dc196673032570950a13343a203fcd5cb94eec934b29ec820209198b0

L3: tipo UsuarioActualDTO

L3: metodo UsuarioActualDTO

Backend/demo/src/main/java/com/example/demo/model/CatalogoOcupacion.java

26 lineas; SHA-256: 3053c17ceb118e39f1ca9776571b7eebefef4aa796c6d6e1e44effe36f7005f2

L7: tipo CatalogoOcupacion

L19: metodo getId

L21: metodo setId

L23: metodo getNombre

L25: metodo setNombre

Backend/demo/src/main/java/com/example/demo/model/CatalogoParentesco.java

25 lineas; SHA-256: fa884333ff21416c855663d84f4bfcbaaf3a0584ea0441bfaf9d3547b14656d6

L9: tipo CatalogoParentesco

L18: metodo getId

L20: metodo setId

L22: metodo getNombre

L24: metodo setNombre

Backend/demo/src/main/java/com/example/demo/model/ContactoEmergencia.java

44 lineas; SHA-256: f627a351382c7adf734f1764a74ca91912814bbd547a6905fbcf7ad19e80f125

L10: tipo ContactoEmergencia

L29: metodo getId

L31: metodo setId

L33: metodo getPersona

L35: metodo setPersona

L37: metodo getContacto

L39: metodo setContacto

L41: metodo getParentesco

L43: metodo setParentesco

Backend/demo/src/main/java/com/example/demo/model/DireccionTitular.java

78 lineas; SHA-256: c1341b885057485f42f30e79f39e4c2520e8d54781d9769c0d04fb5c36328b65

L7: tipo DireccionTitular

L41: metodo getId

L43: metodo getPerfilTitular

L45: metodo setPerfilTitular

L47: metodo getOrden

L49: metodo setOrden

L51: metodo getPais

L53: metodo setPais

L55: metodo getEstado

L57: metodo setEstado

L59: metodo getMunicipio

L61: metodo setMunicipio

L63: metodo getColonia

L65: metodo setColonia

L67: metodo getCodigoPostal

L69: metodo setCodigoPostal

L71: metodo getCalle

L73: metodo setCalle

L75: metodo getNumero

L77: metodo setNumero

Backend/demo/src/main/java/com/example/demo/model/PerfilTitular.java

32 lineas; SHA-256: 20077078eb48cc9a84f3246884cf1732dd7558f17e0e8e327c5bc3a9885c6baa

L10: tipo PerfilTitular

L19: metodo getPersona

L21: metodo setPersona

L23: metodo getOcupacion

L25: metodo setOcupacion

L27: metodo getFechaBaja

L29: metodo setFechaBaja

L31: metodo getDirecciones

Backend/demo/src/main/java/com/example/demo/model/Persona.java

60 lineas; SHA-256: a4ac0e5f3cae24351f9848ed23a2abc9b4b19ed5f965ec2a1c5c676a430175ea

L12: tipo Persona

L23: metodo getId

L25: metodo setId

L27: metodo getNombre

L29: metodo setNombre

L31: metodo getApellido

L33: metodo setApellido

L35: metodo getFechaNacimiento

L37: metodo setFechaNacimiento

L39: metodo getGenero

L41: metodo setGenero

L43: metodo getPerfilTitular

L45: metodo setPerfilTitular

L47: metodo isTitular

L49: metodo setTitular

L51: metodo getFechaBaja

L53: metodo setFechaBaja

L55: metodo getContactosEmergencia

L57: metodo getCorreos

L59: metodo getTelefonos

Backend/demo/src/main/java/com/example/demo/model/PersonaCorreo.java

34 lineas; SHA-256: 8385606bf2dae32cefdfbaa4249991223acf817cf431fb2ce90fa23bc0b083fd

L7: tipo PersonaCorreo

L23: metodo getId

L25: metodo setId

L27: metodo getPersona

L29: metodo setPersona

L31: metodo getCorreo

L33: metodo setCorreo

Backend/demo/src/main/java/com/example/demo/model/PersonaTelefono.java

34 lineas; SHA-256: e3f67b38258d7631bce312618cc7d00280366e8b42868c03e5d66c4eb13cdb84

L7: tipo PersonaTelefono

L23: metodo getId

L25: metodo setId

L27: metodo getPersona

L29: metodo setPersona

L31: metodo getTelefono

L33: metodo setTelefono

Backend/demo/src/main/java/com/example/demo/model/Usuario.java

49 lineas; SHA-256: 97cc7126f230a9c7c922eba3840480a0fd89c2525b60d115d43f3c37d2503917

L7: tipo Usuario

L26: metodo getId

L28: metodo setId

L31: metodo getUsername

L33: metodo setUsername

L36: metodo getPassword

L38: metodo setPassword

L41: metodo getRol

L43: metodo setRol

L46: metodo getIdPersona

L48: metodo setIdPersona

Backend/demo/src/main/java/com/example/demo/repository/CatalogoOcupacionRepository.java

12 lineas; SHA-256: 98b4a44c762a83ab57de7a07ff60f7be281201376de65b92be11d1f924938800

L9: tipo CatalogoOcupacionRepository

Backend/demo/src/main/java/com/example/demo/repository/CatalogoParentescoRepository.java

10 lineas; SHA-256: de09d5761757d58545f9f4a1384bff7efa091b9894d17aeca54fe9cdd64892fc

L7: tipo CatalogoParentescoRepository

Backend/demo/src/main/java/com/example/demo/repository/PersonaRepository.java

94 lineas; SHA-256: 3bb0e2e1b7b4b99de6f93c111cff3ec8a6e094983d937f3cbd46e564cb7adaf9

L12: tipo PersonaRepository

Backend/demo/src/main/java/com/example/demo/repository/UsuarioRepository.java

16 lineas; SHA-256: a3be220b2d8a6223e364fbc1a1aba1512170e3a706c7c30d2bd73bcfc44adb61

L7: tipo UsuarioRepository

Backend/demo/src/main/java/com/example/demo/security/JwtFilter.java

51 lineas; SHA-256: 9b18410e5ebecf7a227c721962bdb92d37a80502768a70c65f97a59b936fd033

L18: tipo JwtFilter

L25: metodo doFilterInternal

Backend/demo/src/main/java/com/example/demo/security/JwtUtil.java

45 lineas; SHA-256: 724a6f1c73f097dfc7790af6a5540779d3482bd6fe65148725d19f33414bcb5f

L15: tipo JwtUtil

L22: metodo getSigningKey

L27: metodo generateToken

L37: metodo extractUsername

Backend/demo/src/main/java/com/example/demo/service/CuentaService.java

79 lineas; SHA-256: 5bccc87ac65c761b2590c0e7c6f6795480168d32623b0a7e6d7c2f0f592619aa

L17: tipo CuentaService

L31: metodo obtenerDatosLogin

L39: metodo actualizarLogin

L67: metodo obtenerUsuarioActual

Backend/demo/src/main/java/com/example/demo/service/PersonaService.java

533 lineas; SHA-256: 188b71bfdbd879ddeefeeb0d10970e24e10ac4a4532c4c5c15e7f46cf4536773

L18: tipo PersonaService

L24: metodo obtenerContactos

L34: metodo contactoDesdeFila

L51: metodo separarComunicaciones

L58: metodo guardarContactos

L84: metodo obtenerDetalle

L92: metodo obtenerPorId

L98: metodo obtenerTodos

L104: metodo obtenerInactivos

L110: metodo reactivar

L127: metodo guardar

L146: metodo actualizar

L177: metodo actualizarParcial

L213: metodo eliminarLogico

L225: metodo convertirADTO

L243: metodo convertirAEntidad

L272: metodo validarTitular

L281: metodo direccionesDe

L287: metodo convertirDirecciones

L295: metodo reemplazarDirecciones

L323: metodo length

L326: metodo valor

L331: metodo titularActivo

L339: metodo invalido

L345: metodo validarContactos

L375: metodo resolverParentesco

L382: metodo reemplazarContactos

L423: metodo normalizarCorreo

L429: metodo buscarContactos

L448: metodo validarComunicacion

L453: metodo validarComunicaciones

L469: metodo verificarCoincidencia

L476: metodo actualizarDatosContacto

L483: metodo actualizarComunicacion

L488: metodo sincronizarComunicaciones

L505: metodo resumirPersonas

L513: metodo convertirAResumen

L525: metodo obtenerResumen

Backend/demo/src/main/resources/application.properties

11 lineas; SHA-256: e554a6d068c355f0845cd3f8dddaf01e6e0947fb0b8b9bce34866554f61665c2

Backend/demo/src/main/resources/data.sql

46 lineas; SHA-256: 2120fc1d063ec9109674f0521a984346173caa530accab5200db708911750f75

Backend/demo/src/main/resources/schema.sql

70 lineas; SHA-256: 902f40c1b37137d8ac05185431d3dba791c230e2951e1e8fd7b8462d6b668bdf

L1: tabla catalogo_ocupacion

L2: tabla catalogo_parentesco

L5: tabla persona

L14: tabla perfil_titular

L21: tabla direccion_titular

L35: tabla persona_correo

L41: tabla persona_telefono

L47: tabla persona_contacto_emergencia

L59: tabla usuario

Backend/demo/src/test/java/com/example/demo/DemoApplicationTests.java

173 lineas; SHA-256: fc2a8f02e973bd98da9f05afc9e78e361f05736287f35aa2c4892519d1ec7ee2

L28: tipo DemoApplicationTests

L37: metodo formulario

L48: metodo guardarConDatos

L54: metodo existente

L55: metodo count

Backend/demo/src/test/java/com/example/demo/security/JwtUtilTest.java

21 lineas; SHA-256: b4fa3c0d32d70d997b5b82cd7509a8c658f2dc2a637a92d59008e6bec8a0cfdb

L10: tipo JwtUtilTest

Backend/demo/src/test/java/com/example/demo/service/CuentaServiceTest.java

109 lineas; SHA-256: 562b3e3968b749ff226a8db20db275939d284f067be9e8c2f16923d198a56465

L19: tipo CuentaServiceTest

L20: metodo cuentaEditable

Backend/demo/src/test/java/com/example/demo/service/PersonaContratosTest.java

84 lineas; SHA-256: af4a4e79dca3fe6a43c68609ad021dfb329c38d8c3f424ec14ef20930d6dc77e

L12: tipo PersonaContratosTest

Backend/demo/src/test/java/com/example/demo/service/RegistroCompletoTest.java

404 lineas; SHA-256: d5a0586e0cc19f48659053926185823e8a4841a974a65d3dffeaa5c44ff4d698

L15: tipo RegistroCompletoTest

L90: metodo guardarConDatos

L133: metodo nuevo

L136: metodo existente

L139: metodo datos

L151: metodo persona

L157: metodo relacion

Frontend/README.md

99 lineas; SHA-256: 303416541aa99bb468b94a616d9dd499b737136b5d30800ddcc4e966bbab4ae7

L1: seccion Frontend Angular de administración de personas

L5: seccion Ejecución

L18: seccion Entradas y organización

L36: seccion Rutas

L56: seccion Contratos y formularios

L68: seccion Servicios, cachés y errores

L83: seccion Tema y presentación

L91: seccion Verificación

README.md

75 lineas; SHA-256: 90b677fe7d0fb4570c5f6f05ec195bd51946f39b3847ee918ab6b6f9da13144c

L1: seccion Sistema de administración de personas

L5: seccion Documentación técnica

L22: seccion Organización del repositorio

L43: seccion Tecnologías comprobadas

L56: seccion Inicio local

L65: seccion Modelo funcional vigente

docs/API.md

147 lineas; SHA-256: d5809cf402071a7e5a88b977f29046edea24ae08bafd27493c0031c47213b6c3

L1: seccion Contrato HTTP vigente

L3: seccion Convenciones

L9: seccion Endpoints

L37: seccion DTOs de lectura

L78: seccion Alta y actualización de titular

L100: seccion Contactos compartidos y sustitución de relaciones

L116: seccion Coincidencias

L120: seccion Cuenta y login

L126: seccion Errores y límites HTTP

L141: seccion Edicion parcial del titular

docs/ARQUITECTURA.md

112 lineas; SHA-256: e483eb8df1d74a0d4b0dcc529a7525d8bb4e0fc1e41fc8bd55a263de8348eaa5

L1: seccion Arquitectura y flujos internos

L3: seccion 1. Topología

L18: seccion 2. Frontend

L26: seccion 3. Backend

L40: seccion 4. Persistencia y transacciones

L52: seccion 5. Alta y edición

L79: seccion 6. Autenticación y resiliencia

L91: seccion 7. Política de cachés

L106: seccion 8. Diseño y accesibilidad

docs/BASE_DE_DATOS.md

123 lineas; SHA-256: 2f84b2ee876dd1709b2bfd2d6f9acbcc23c49d498d039d79d7215fe9f018fad4

L1: seccion Modelo PostgreSQL y persistencia JPA

L3: seccion Fuente de verdad

L9: seccion Relaciones

L26: seccion Diccionario de tablas

L28: seccion `persona`

L39: seccion `perfil_titular`

L49: seccion `direccion_titular`

L66: seccion Comunicaciones

L75: seccion `persona_contacto_emergencia`

L86: seccion Catálogos y usuario

L98: seccion Índices declarados

L109: seccion Semilla y secuencias

L119: seccion DDL automático y alcance de verificación

docs/OPERACION_Y_PRUEBAS.md

166 lineas; SHA-256: 55e004204e68535a1dd2600c3a33837da8326bbba9ec89501d7f78b869de7003

L1: seccion Instalación, operación y verificación

L3: seccion Requisitos y versiones

L13: seccion 1. Crear el esquema de desarrollo

L26: seccion 2. Configurar y arrancar el backend

L43: seccion Desde Backend/demo

L49: seccion 3. Instalar y arrancar el frontend

L52: seccion Desde Frontend

L61: seccion 4. Compilación

L76: seccion 5. Pruebas frontend

L86: seccion 6. Pruebas backend

L89: seccion Desde Backend/demo

L114: seccion Integración PostgreSQL, solo sobre una base desechable

L126: seccion Configurar TEST_DATABASE_USERNAME y TEST_DATABASE_PASSWORD en la sesión según la instancia.

L132: seccion 7. Contraseñas y documentación

L150: seccion 8. Diagnóstico

L164: seccion Alcance de los resultados

docs/REVISION_TECNICA.md

55 lineas; SHA-256: 17949023df0f3b40d0f8fcbcbade45ba1f4f2272ab43e702726284aa1658c552

L1: seccion Revisión técnica y limitaciones conocidas

L5: seccion Resultados comprobados en la revisión base (2026-10-15)

L13: seccion Hallazgos

L38: seccion Diferencias entre requisitos de captura y backend

L49: seccion Cobertura y tareas pendientes de validación

docs/back/BACKEND_ESPECIFICACION_AGENTES.md

30 lineas; SHA-256: 6a25a5935346618481a4335b0692aefc7879f0e887be2001579f06d16abd371f

L1: seccion Contexto técnico backend

L5: seccion Stack/capas

L9: seccion Modelo y contratos

L24: seccion Pruebas/documentación

docs/back/generar_guia_backend.py

189 lineas; SHA-256: fa6120546227f9f00ff638fdd66dbbcc322598a7bd8fa0bcd487b46bf0befedd

L32: funcion source_paths

L46: funcion declarations

L65: funcion inventory

L77: funcion plain

L83: funcion styles

L100: funcion append_markdown

L119: funcion page_footer

L127: funcion generate_pdf

L159: funcion main

docs/front/FRONTEND_ESPECIFICACION_AGENTES.md

32 lineas; SHA-256: d8e3f2cfe5e3d8818decf7bb36f13514c29578a54c06bba480e73db3cc52a82b

L1: seccion Contexto técnico frontend

L5: seccion Stack y ejecución

L9: seccion Invariantes

L18: seccion Estado y errores

L26: seccion UI y verificación